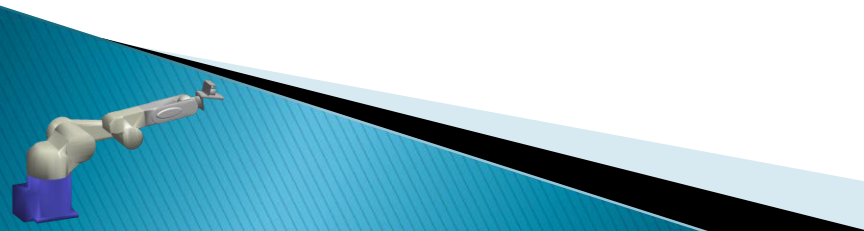


Week 5

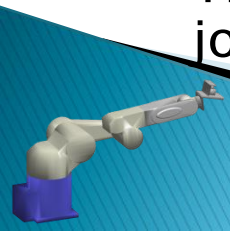
Schedule

- ▶ Quiz 2:
 - Individual (30+10=40 mins)
 - Buffer (5 mins)
- ▶ Assignment 1 updates
- ▶ Review & discuss Lab 4 (15 mins)
- ▶ Introduction to Lab 5 (5 mins)
- ▶ Collaboratively work on Labs or Assignment 1 (≈ 2 hours)



Quiz 2 Hints (see Teams)

- ▶ Focus on tools from Week 3 & 4 (DH parameters, forward kinematics, trajectory planning).
- ▶ **Hint 1:** Be ready to create and calculate forward kinematics on Baxter robot's left and right arms.
- ▶ **Hint 2:** Know how to calculate the distance between two 3D points (and that after setting the model.base then the end-effector to ground distance is the Z value).
- ▶ **Hint 3:** Understand how to calculate the rotational distance from a joint state to upper and lower limits. (always positive)
- ▶ **Hint 4:** Be able to generate Quintic Polynomial (jtraj) and Trapezoidal Velocity profiles, and find the maximum absolute joint velocity.



Week 5 – Quiz 2

Attendance

- In-class? Display student card and tutors will mark you off

- ▶ Open book Quiz with no communicating
- ▶ $30+10=40$ mins
- ▶ ~10 Questions
- ▶ Individual Access Code =
Baxter

Upload Code after the quiz
copy/paste



Weeks 5–6 Plans

August 2025

Mo	Tu	We	Th	Fr	Sa	Su
28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

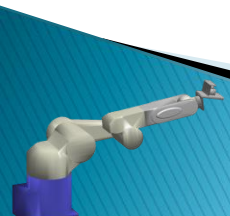
Week 5 Class Quiz 2

Supervised Lab Drop-in
CB.11.10.403
To use the UR3e robots
9am–3pm

Assignment 1 –
safety
documentation
deadline (tonight)
*... but, please get
it checked in class
by a tutor today
27/8*

Last chance to demo
a real UR3e robots for
marks

Assignment 1 – Viva (online)
Bookings required (see Canvas)



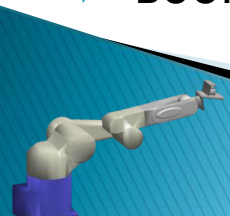
Lab Assignment 1

Task	Weight	Due
1. Safety Documentation	4%	Marked by a tutor in week 4 or 5 class, then the documentation is submitted to Canvas by 21:00 Wednesday 25th August 2025
2. Real Robot Control from Matlab/Python	2%	Marked off by a tutor in class: Week 5 drop-in lab or Week 5 class or the Week 6 drop-in lab.
3. Online Demo & Code Viva	14%	Week 6 (Wednesday 3rd September 2025) Approximately 20 minutes online during normal class time – bookings required

- ▶ Submit Safety Documentation to Canvas (RA & SWMS). Ensure
 - All fields in the form are filled in and signed (typed name is fine)
 - Your RA shows only L-level residual risk.
 - SWMS includes a paragraph describing the task at the top.
 - Proposed PPE and hazards are sensible and realistic.
 - SWMS has detailed steps in the work procedure section (bottom).
 - Talk to a tutor – get marked off in-class!
- ▶ Do your "FEIT CB11 Mechatronics Lab Induction"
<https://inductions.uts.edu.au/>
- ▶ Supervised Lab Drop-in CB.11.10.403
to use the real robots 9am–3pm on Monday 1/9
- ▶ Book a Viva marking timeslot for Week 6 →
or apply for extension via S/C or A/S



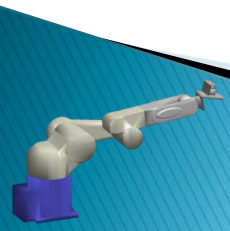
Viva booking links in Canvas



LinearUR3 for Assignment 1

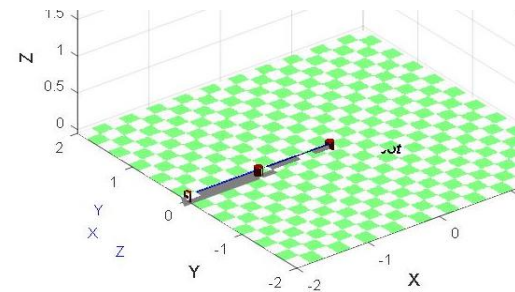
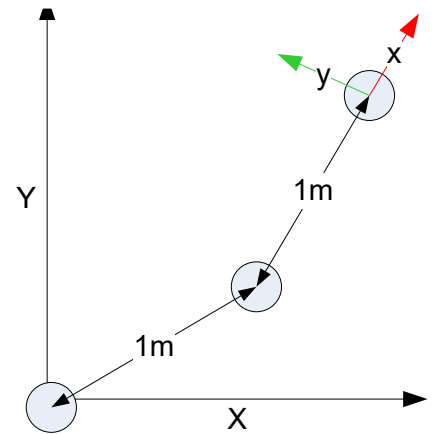
- ▶ Tutors reviewed UR3, UR5 and LinearUR5 models in ir_support
- ▶ UR3/UR5 models follow official DH parameters from Universal Robots; LinearUR5 does not.
- ▶ A rough LinearUR3 model is now provided and approved for use without penalty.
- ▶ Based on UR3 DH parameters with small d (z-translation) adjustments (<1 cm) for rail height differences.
- ▶ Accurate enough for assignment purposes (not photorealistic).
- ▶ If you already built your own model, you may continue using it; this will also help for Assignment 2.
- ▶ To use: upgrade to latest ir_support (v1.2.6) using:

```
pip install --force-reinstall --no-deps --no-cache-dir ir-support
```



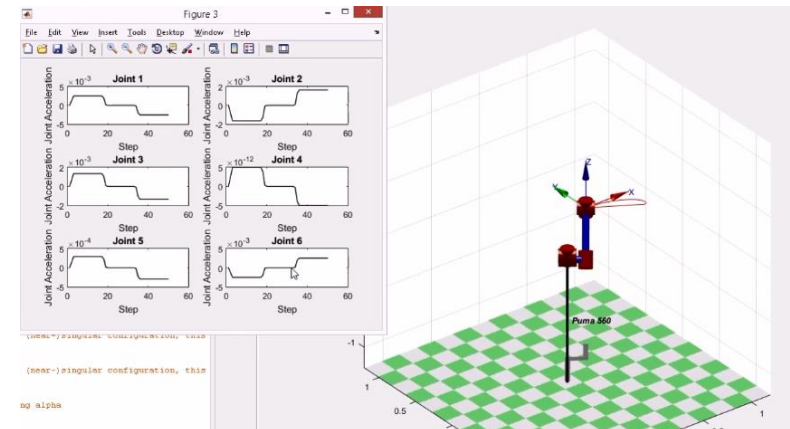
Review of Lab 4 – Q1

- ▶ Draw with 3DOF Planar Arm so the Z axis faces down ways
- ▶ What did you observe when using “teach” to move the robot?
- ▶ Use ikine and mask out the impossible-to-alter values
- ▶ Using ikine draw a line. What was observed about its “straightness”?
- ▶ Trace out a circle using ikine. What was observed about limits and collisions?



Review of Lab 4 – Q2 and Q3

- ▶ Q2: Inverse Kinematics to Determine Joint Angles of Puma 560.
 - Did you have any cases that didn't work?
 - What error messages?
 - Did you get a joint angle returned that didn't allow the robot to be at the requested transform
 - Does ikine obey joint limits?
 - What other inverse kinematic solver is in the toolbox? Does it obey joint limits?
- ▶ Q3 Interpolation: Get from A to B using puma560. What was learned from this exercise about:
 - Quintic Polynomial Profile
 - Trapezoidal Velocity Profile

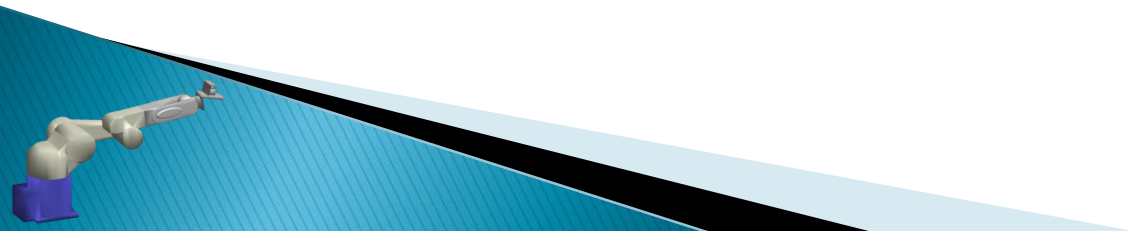


Introduction to Lab 5

- ▶ Q1: Help the Robot Blaster save the Planet from UFOs!
Develops understanding of
 - Line-plane intersections
 - Inverse kinematics
 - Gamifying Matlab
 - Note: Lab5SolutionQuestion1Skeleton.m
- ▶ Q2: Collision **checking** for 3-link planar robot
- ▶ Q3: Collision **avoidance** for 3-link planar robot

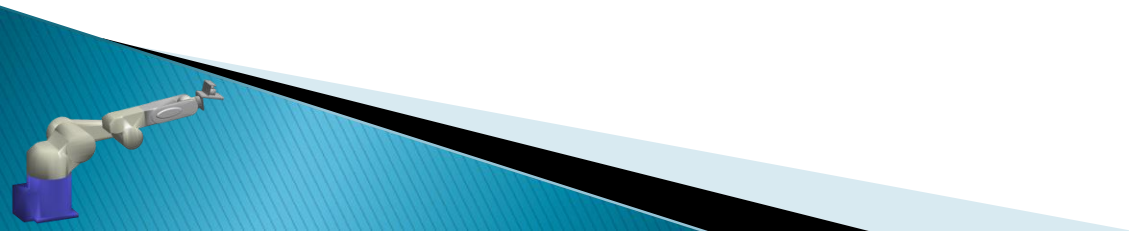
Pre-work before Week 6 lab

- ▶ Completed week 5 lab exercises
- ▶ Ensure you have achieved 80% in Quizzes 1&2
- ▶ Read textbook
 - Section 3.2 (pages 51–56): “Time Varying Coordinate Frames”
 - Appendix E (pages 517–522): “Ellipses”
- ▶ Watch videos on Canvas
- ▶ Complete Lab Assignment #1 (due next week)



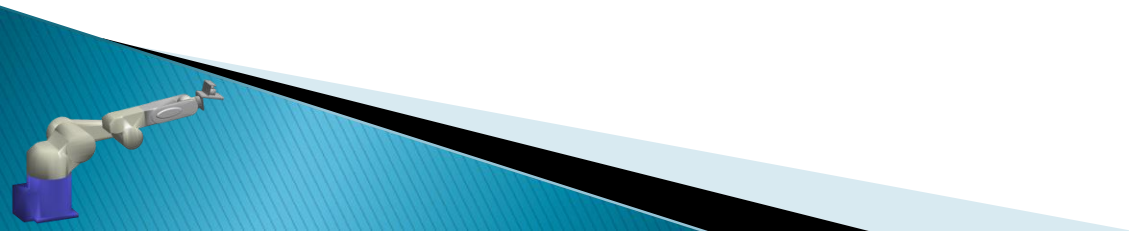
Note/slides from the textbook

- ▶ Textbook readings:
 - Section 3.2 (pages 51–56): “Time Varying Coordinate Frames”
 - Appendix E (pages 517–522): “Ellipses”
- ▶ Although it is better to read the textbook, some notes (in slide format) have been summarised below



Time Varying Coordinate Frames

- ▶ The translational velocity is the rate of change of the position of the origin of the coordinate frame. Rotational velocity is a little more complex.

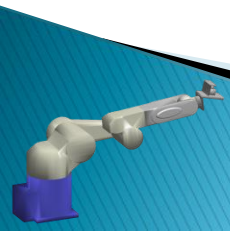


Rotating Coordinate Frame

- ▶ A body rotating in 3-dimensional space has an angular velocity which is a vector quantity

$$\omega = (\omega_x, \omega_y, \omega_z).$$

- ▶ The direction of this vector defines the instantaneous axis of rotation, that is, the axis about which the coordinate frame is rotating at a particular instant of time.
- ▶ The magnitude of the vector is the rate of rotation about the axis – in this respect it is similar to the angle-axis representation for rotation.



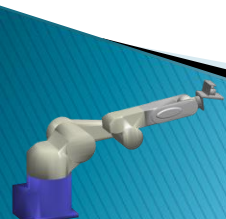
Rotating Coordinate Frame (continued...)

- ▶ From mechanics there is a well known expression for the derivative of a time-varying rotation matrix:

$$\dot{\mathbf{R}} < t > = \mathbf{S}(\boldsymbol{\omega})\mathbf{R} < t >$$

- ▶ Where $\mathbf{R} < t > \in SO(2)$ or $SO(3)$ and $\mathbf{S}(\cdot)$ is a skew-symmetric matrix that, for the 3-dimensional case, has the form

$$\mathbf{S}(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$



Rotating Coordinate Frame (continued...)

- ▶ we might ask what does $\dot{\mathbf{R}}$ mean?

$$\dot{\mathbf{R}} \approx \frac{\mathbf{R} \langle t + \delta_t \rangle - \mathbf{R} \langle \delta_t \rangle}{\delta_t}$$

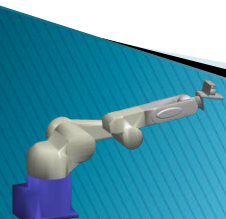
- ▶ which we rearrange as

$$\mathbf{R} \langle t + \delta_t \rangle \approx \delta_t$$

- ▶ and substituting Eq. 3.4 we obtain

$$\mathbf{R} \langle t + \delta_t \rangle \approx \delta_t \mathbf{S}(\omega) \mathbf{R} \langle t \rangle \approx (\delta_t \mathbf{S}(\omega) + \mathbf{I}_{3 \times 3}) \mathbf{R} \langle t \rangle$$

- ▶ which describes how the orthonormal rotation matrix changes as a function of angular velocity.



Incremental Motion

- ▶ Consider a coordinate frame that undergoes a small rotation from R_0 to R_1 . We can write as:

$$R_1 = \delta_t S(\omega) + I_{3 \times 3} R_0$$

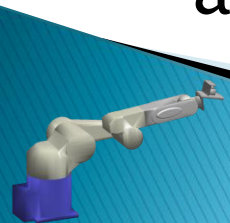
- ▶ and arrange as:

$$\delta_t S(\omega) = R_1 R_0^T - I_{3 \times 3}$$

- ▶ and then apply the vex operator, the inverse of $S(\cdot)$, to both sides

$$\delta_\theta = \text{vex}(R_1 R_0^T - I_{3 \times 3})$$

- ▶ where $\delta_\theta = \delta_t \omega$ a 3-vector with units of angle that represents an infinitesimal rotation about the world x -, y - and z -axes.



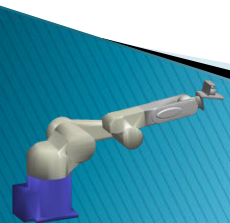
Incremental Motion (continued...)

- ▶ Given two poses ξ_0 and ξ_1 that differ infinitesimally we can represent the difference between them as a 6-vector

$$\delta = \Delta(T_0, T_1) = \begin{bmatrix} t_1 - t_0 \\ \text{vec}(\mathbf{R}_1 \mathbf{R}_0^T - \mathbf{I}_{3 \times 3}) \end{bmatrix}$$

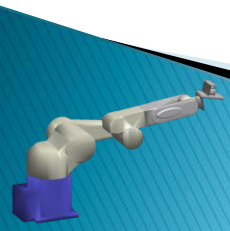
- ▶ where $T_0 = (R_0, t_0)$ and $T_1 = (R_1, t_1)$
- ▶ The inverse operation is: $\xi = \Delta^{-1}(\delta)$
- ▶ and for homogenous transformation representation is:

$$T = \begin{bmatrix} S(\delta_\theta) & \delta_d \\ 0_{3 \times 1} & 0 \end{bmatrix} + I_{3 \times 3}$$



Inertial Navigation Systems

- ▶ An inertial navigation system is a “black box” that estimates its velocity, orientation and position with respect to the inertial reference frame (the universe).
- ▶ It has no external inputs such as radio signals from satellites and this makes it well suited to applications such as submarine, spacecraft and missile guidance.
- ▶ An inertial navigation system works by measuring accelerations and angular velocities and integrating them over time.



Inertial Navigation Systems (continued...)

- ▶ A discrete-time such as:

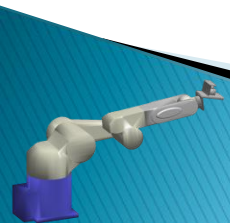
$$\mathbf{R}^{<k+1>} = \delta_t \mathbf{S}(\boldsymbol{\omega}) \mathbf{R}^{<k>} + \mathbf{R}^{<k>}$$

is used to numerically integrate changes in pose in order to estimate the orientation of the vehicle.

- ▶ The measured acceleration B_a of the vehicle's body frame is rotated into the inertial frame

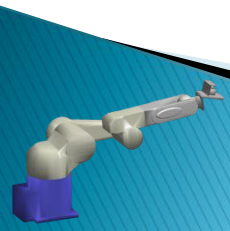
$$0_a = 0_{R_B} B_a$$

and can then be integrated twice to update the estimate of the vehicle's position in the inertial frame.



Ellipses

- ▶ An ellipse belongs to the family of planar curves known as conics.
- ▶ The simplest form of an ellipse is defined implicitly: $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$



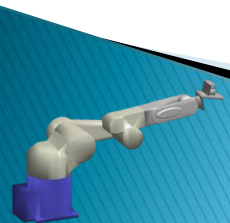
Ellipses (continued...)

- ▶ We can write the ellipse in matrix quadratic form as:

$$(x \quad y) \begin{bmatrix} 1/a^2 & 0 \\ 0 & 1/b^2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 1$$

$$x^T \begin{bmatrix} a^2 & 0 \\ 0 & b^2 \end{bmatrix}^{-1} x = 1$$

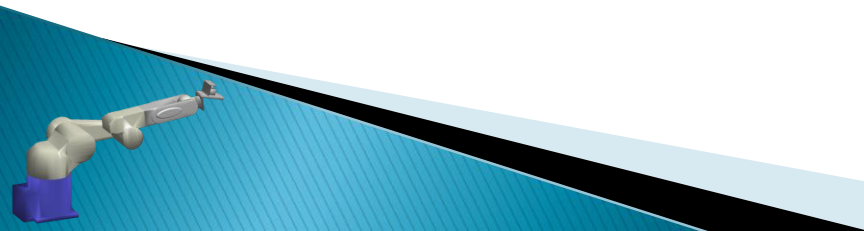
$$x^T E^{-1} x = 1$$



Ellipses (continued...)

- ▶ In the most general form E is a **symmetric matrix** $E = \begin{bmatrix} A & C \\ C & B \end{bmatrix}$
- ▶ and its determinant $\det(E) = AB - C^2$ defines the type of conic

$$\det(E) \begin{cases} > 0 \text{ ellipse} \\ = 0 \text{ parabola} \\ < 0 \text{ hyperbola} \end{cases}$$



Ellipses (continued...)

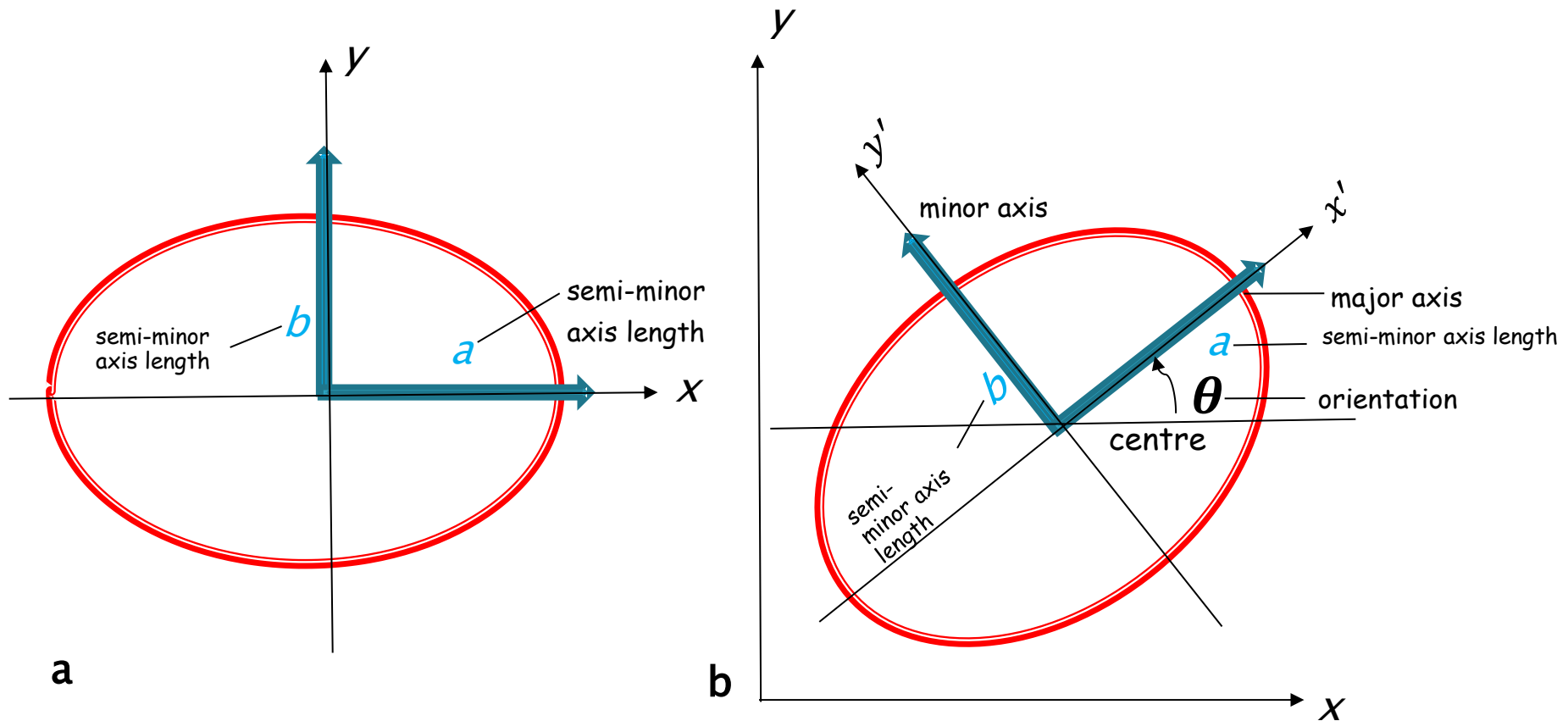


Fig. E.1. Ellipses. **a** Canonical ellipse centred at the origin and aligned with the x - and y -axes; **b** general form of ellipse

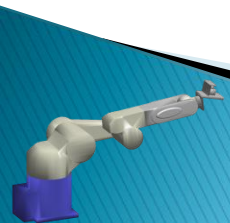
Ellipses (continued...)

- ▶ An ellipse is therefore represented by a positive definite symmetric matrix E .
- ▶ Non-zero values of C change the orientation of the ellipse. The ellipse can be arbitrarily centred at x_c by writing it in the form $(x - x_c)^T E^{-1} (x - x_c) = 1$
- ▶ which leads to the general ellipse $E = X\Lambda X^T$
- ▶ where X is an orthogonal matrix comprising the eigenvectors of E . The inverse is $E^{-1} = X\Lambda^{-1}X^T$
- ▶ so the quadratic form becomes

$$x^T X\Lambda^{-1}X^T x = 1$$

$$(X^T x)^T \Lambda^{-1} (X^T x) = 1$$

$$x'^T \Lambda^{-1} x' = 1$$



Ellipses (continued...)

- ▶ Alternatively the ellipse can be represented in polynomial form.

$$(x - (x_0, y_0))^T \begin{bmatrix} a & c \\ c & b \end{bmatrix} (x - (x_0, y_0)) = 1$$

- and expand we obtain

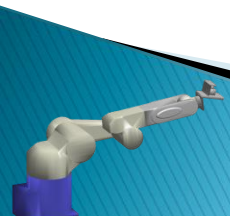
$$e_1 x^2 + e_2 y^2 + e_3 xy + e_4 x + e_5 y + e_6 = 0$$

- where $e_1 = a, e_2 = b, e_3 = 2c, e_4 = -2(ax_0 + cy_0),$
 $e_5 = -2(by_0 + cx_0)$

$$\text{and } e_6 = ax_0^2 + by_0^2 + 2x_0y_0 - 1$$

- ▶ For a non-degenerate ellipse $e_1 \neq 0$ and we rewrite the polynomial in normalized form

$$x^2 + E_1 y^2 + E_2 xy + E_3 x + E_4 y + E_5 = 0$$



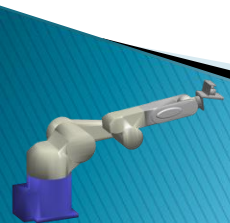
Properties

- ▶ The area of an ellipse is πab and its eccentricity is

$$\varepsilon = \frac{\sqrt{a^2 - b^2}}{a}$$

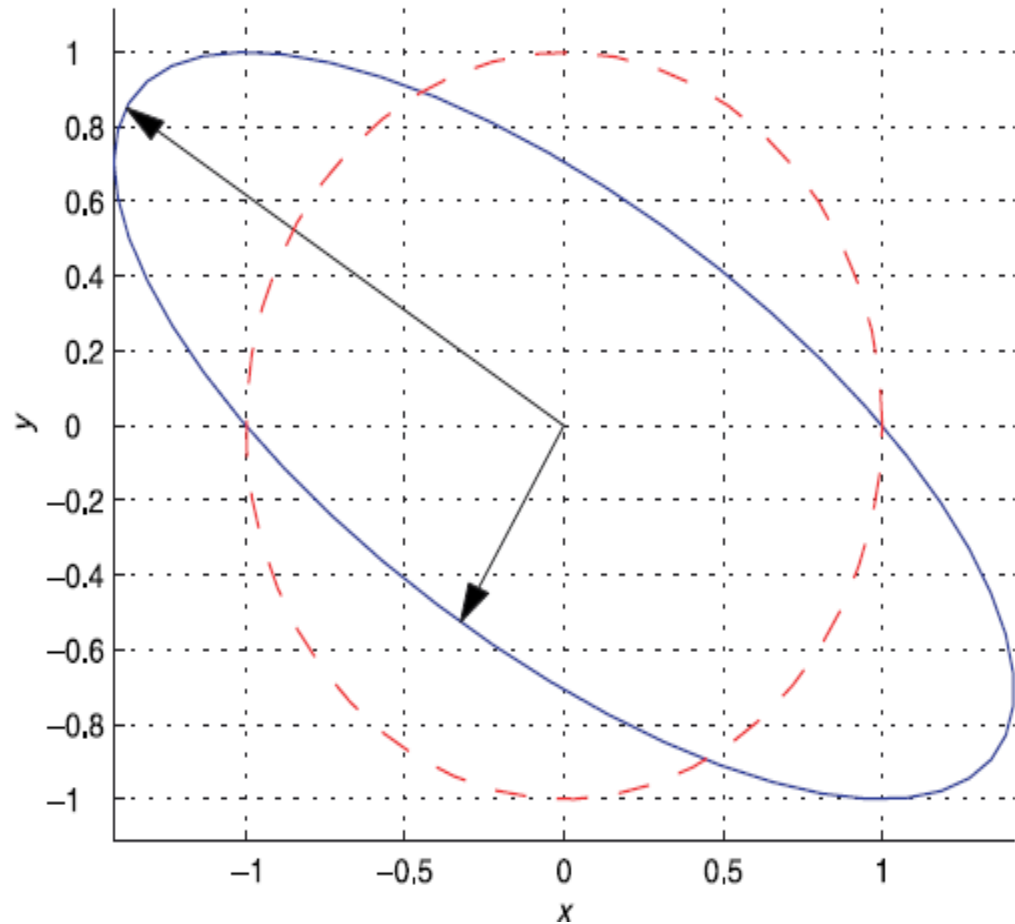
- ▶ The eigenvectors of $\mathbf{E}$ define the principal directions of the ellipse and the square root of the eigenvalues are the corresponding radii.
- ▶ Consider the ellipse

$$x \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}^{-1} x = 1$$



Properties (continued...)

- ▶ **Fig. E.2.** Ellipse corresponding to symmetric 2×2 matrix, and the unit circle shown in red. The arrows indicate the major and minor axes of the ellipse



Drawing an Ellipse

- ▶ In order to draw an ellipse we first define a point $y = [x, y]^T$ on the unit circle $y^T y = 1$ and rewrite as $x^T E^{-\frac{1}{2}} E^{-\frac{1}{2}} x = 1$

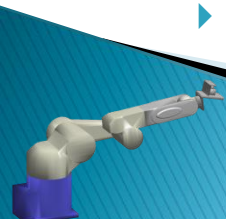
- where $E^{\frac{1}{2}}$ is the matrix square root
- Equating these two equations we can write

$$x^T E^{-\frac{1}{2}} E^{-\frac{1}{2}} x = y^T y$$

- It is clear that $y = E^{-\frac{1}{2}} x$ which we can arrange as: $x = E^{\frac{1}{2}} y$
- which transforms a point on the unit circle to a point on an ellipse.
- If the ellipse is centered at x_c rather than the origin we can perform a change of coordinates:

$$(x - x_c)^T E^{-\frac{1}{2}} E^{-\frac{1}{2}} (x - x_c) = 1$$

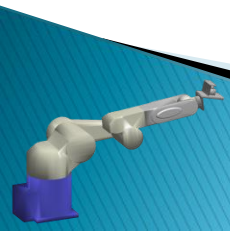
- ▶ from which we write the transformation as: $x = E^{\frac{1}{2}} y + x_c$



Fitting an Ellipse to Data From a Set of Interior Points

- ▶ A common approach is to find the ellipse that has the same mass properties as the set of points.
- ▶ From the set of N points $x_i = (x_p, y_i)$ we can compute the moments

$$\begin{aligned}m_{00} &= N \\m_{10} &= \sum_{i=1}^N x_i \\m_{01} &= \sum_{i=1}^N y_i\end{aligned}$$



Fitting an Ellipse to Data

From a Set of Interior Points (continued...)

- ▶ The centre of the ellipse is taken to be the centroid of the set of points

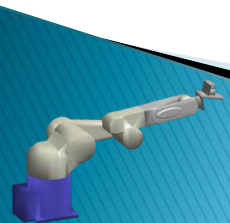
$$(x_c, y_c) = \left[\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right]$$

- which allows us to compute the central second moments

$$\mu_{20} = \sum_{i=1}^N (x_i - x_c)^2$$

$$\mu_{02} = \sum_{i=1}^N (y_i - y_c)^2$$

$$\mu_{20} = \sum_{i=1}^N (x_i - x_c)(y_i - y_c)$$



Fitting an Ellipse to Data

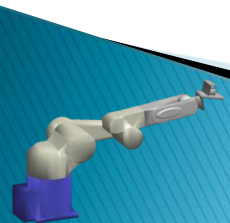
From a Set of Interior Points (continued...)

- ▶ The inertia matrix for a general ellipse is the symmetric matrix

$$J = \begin{bmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{bmatrix}$$

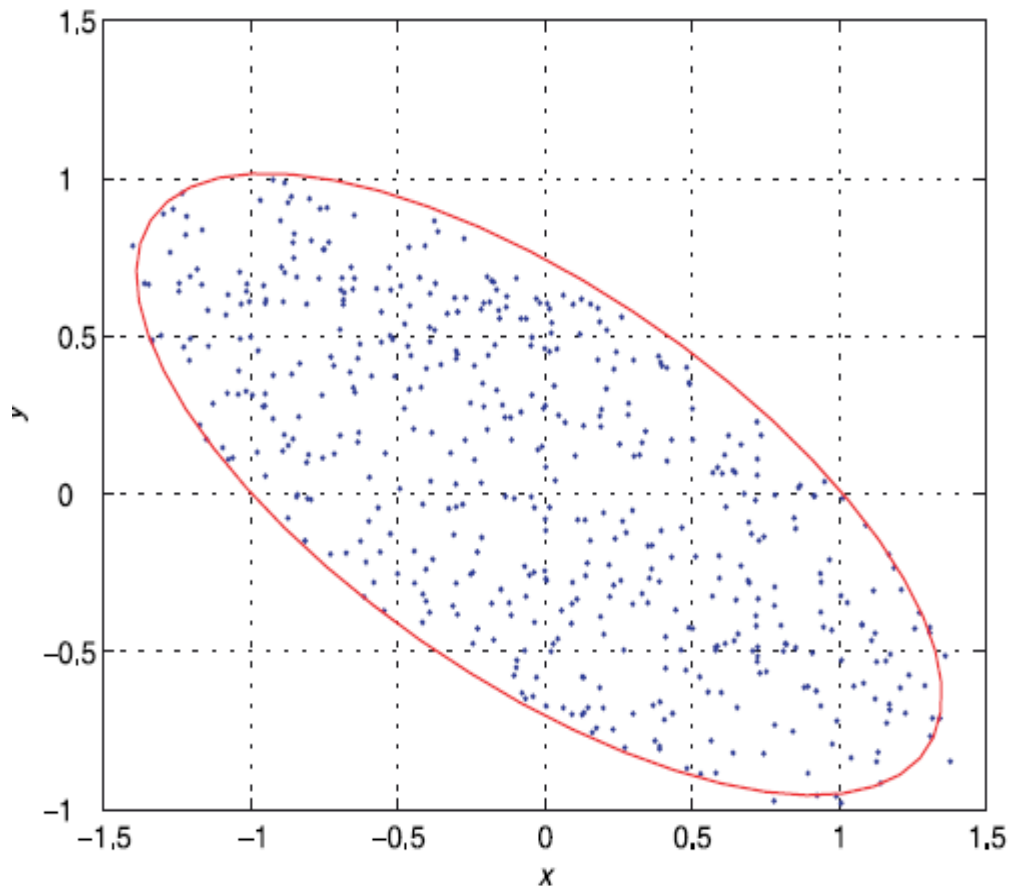
- ▶ where the diagonal terms are the moments of inertia and the off-diagonal terms are the products of inertia. Inertia can be computed more directly by

$$J = \sum_{i=1}^N (x - x_c)(x - x_c)^T$$



Fitting an Ellipse to Data From a Set of Interior Points (continued...)

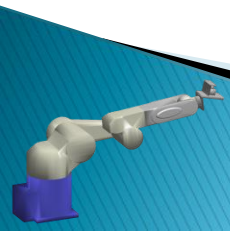
► Fig. E.3. Point data



Fitting an Ellipse to Data From a Set of Interior Points (continued...)

- ▶ The relationship between the inertia matrix and the symmetric ellipse matrix is

$$E = \frac{4}{m_{00}} J$$

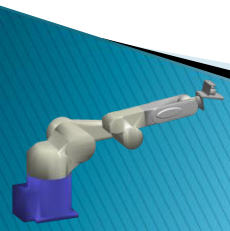


From a Set of Boundary Points

- ▶ Using the polynomial form of the ellipse for each point we write this in matrix form

$$\begin{bmatrix} y_1^2 & x_1 y_1 & x_1 & y_1 & 1 \\ y_2^2 & x_2 y_2 & x_2 & y_2 & 1 \\ \vdots & & & & \\ y_N^2 & x_N y_N & x_N & y_N & 1 \end{bmatrix} \begin{bmatrix} E_1 \\ E_2 \\ E_3 \\ E_4 \\ E_5 \end{bmatrix} = \begin{bmatrix} -x_1^2 \\ -x_2^2 \\ \vdots \\ -x_N^2 \end{bmatrix}$$

and for $N \geq 5$ we can solve for the ellipse parameter vector.



References

- ▶ [1] Robotics, Vision and Control. Peter Corke